Semantic Early-Stopping for Iterative LLM Agent Loops: A Judge-Efficient Study of *When to Halt*

Sahil Shrivastava
Semantic Halting Problem (SHP) Project
Open implementation, machine-checked theory, reproducible harness
`github.com/SahilShrivastava-Dev/semantic-halting-problem`

***Abstract*—Multi-agent large language model (LLM) loops—for example a Writer that drafts and a Critic that revises—are almost always terminated by a fixed iteration cap (`max_iterations`). This is a *syntactic* kill-switch: it is blind to whether the answer is still improving, so it over-spends tokens on easy inputs and truncates hard ones. We study semantic early-stopping: the loop halts when consecutive draft embeddings stop changing in meaning (cosine-distance with a patience window) and the answer's measured quality stops improving. Our work makes three contributions. First, an honest theoretical footing: we prove deterministic termination and well-definedness and we *machine-check* these claims, while treating the convergence of the distance sequence as an empirically tested conjecture rather than a (previously over-claimed) Banach contraction. Second, a judge-efficient evaluation protocol: we generate each question's full trajectory once, replay every stopping policy over the identical drafts, and cache every LLM-judge call, which yields a strictly paired efficiency-versus-quality comparison at low cost; we further separate operational tokens (charged to a policy) from evaluation tokens (a measurement instrument). Third, an empirical study on multi-hop retrieval-augmented question answering (HotpotQA). On the development split, a judge-free semantic stopper reduces operational tokens by 37% relative to `max_iterations` at parity quality, whereas the full quality-gated variant is counter-productive because its per-round judging dominates cost. An oracle that selects the best round attains $+0.13$ Information Score over every practical policy ($p \approx 3 \times 10^{-5}$), which reframes the problem from "when to stop" (easy) to "which round is best" (open).**

***Index Terms*—LLM agents, early stopping, semantic convergence, retrieval-augmented generation, evaluation methodology, multi-agent systems.**

I. Introduction

ITERATIVE "agentic" patterns are now a standard way to push LLM output quality beyond a single forward pass. The canonical instance is the **Writer→Critic loop**: a Writer produces an answer, a Critic critiques it, the Writer revises, and the cycle repeats until some stopping rule fires. The quality of the final answer depends not only on the two models but, critically, on *when the loop is allowed to stop*.

In practice that decision is made by a single integer: `max_iterations`. The loop runs a fixed number of rounds and then returns whatever it last produced. This is convenient but blunt in two opposite ways. On *easy* inputs the loop reaches a good answer in one or two rounds yet keeps spending tokens, latency, and money until the counter expires. On *hard* inputs the counter may expire before the answer is finished. Worst of all, an integer counter is fundamentally *blind to content*: it cannot perceive that rounds four, five, and six all expressed essentially the same thing. The stopping decision is made on the *iteration index*, never on *what was actually written*.

a) The idea.: We replace the counter with a **content-aware** rule. We map each draft to a sentence embedding and measure the **cosine distance** between consecutive drafts. When that distance stays below a small threshold ε for k consecutive rounds, the answer has **converged in meaning**; additional iteration is churn, and we halt. Because a fast convergence is not the same as a *good* convergence, we pair this geometric signal with a quality signal—an **Information Score** (IS) computed from retrieval-augmented generation (RAG) metrics—so the loop only halts when the answer has both stopped **changing** and stopped **improving**. Two further conditions, an explicit critic approval and a hard failsafe, complete a four-level priority cascade.

b) The honest story behind this paper.: An earlier version of this project asserted that the Writer→Critic update is a *Banach contraction* with a guaranteed unique fixed point. It is not: LLM generation has no proven Lipschitz constant below one and is not deterministic across API calls, so that theorem was unsupported. Rather than dress an idea in mathematics it does not earn, we kept only the claims we can prove (and machine-check) and demoted the rest to measured conjectures. This honesty is itself a contribution: it is the difference between a result a reviewer can trust and one they reject on sight.

c) Contributions.:

1) **Honest, machine-checked theory** (§V): deterministic termination, well-definedness, and halt-priority consistency are proven and verified in code; semantic non-expansiveness is a measured conjecture.

2) **A judge-efficient evaluation protocol** (§VI): trajectory replay, cached judging, and an operational-versus-evaluation token model that exposes hidden measurement cost.
3) **An empirical study** (§VIII) on HotpotQA against five baselines with paired statistics and non-inferiority testing, including the surprising finding that the full quality-gated policy is counter-productive.

II. Background and Related Work

a) Early stopping and adaptive computation.: Stopping a process when a monitored quantity plateaus is classical in machine learning (early stopping on a validation curve) and in adaptive-computation networks (early-exit inference). SHP transports the same intuition to the *outer* loop of an agent system, where the monitored quantity is the *semantic* change between successive natural-language drafts rather than a scalar loss.

b) Semantic convergence and fixed points.: Several lines of work study convergence of iterative operators in embedding space and the termination of multi-agent systems. SHP shares the convergence intuition but differs in two respects: it refuses the unsupported contraction guarantee, proving only termination; and it *gates* convergence on output quality.

c) Uncertainty and orchestration in multi-LLM systems.: Related efforts quantify cross-model disagreement at a single step, or choose *which* orchestration topology to use. These are orthogonal to SHP, which decides *when to exit* a fixed sequential topology based on content.

d) RAG evaluation.: We use RAGAS-style reference metrics (faithfulness, answer relevancy, context precision, context recall) as the quality signal. A recurring caveat—that these metrics are themselves produced by an LLM and are therefore noisy—motivates our use of a stronger judge on the frozen split and of non-inferiority testing rather than equality testing.

(A detailed, per-paper comparison table—method extraction, gap analysis, and cost/benefit—is provided in Appendix A and will be finalized against verified citations.)

III. Problem Formulation

Definition 1 (Scenario)**.** *A scenario is a tuple (q, C, g) with question q, retrieved context passages $C = \{c_1, \ldots, c_n\}$, and ground-truth answer g.*

The loop produces drafts $x_1, x_2, \ldots$. A frozen embedding map ϕ (a local sentence-embedding model) sends each draft to $e_t = \phi(x_t) \in \mathbb{R}^{384}$, L_2-normalized. We define the per-round **semantic distance**

$$d_t \;=\; 1 - \cos(e_t, e_{t-1}) \;=\; 1 - \frac{\langle e_t, e_{t-1}\rangle}{\|e_t\|\,\|e_{t-1}\|} \in [0, 2], \quad t \geq 2. \tag{1}$$

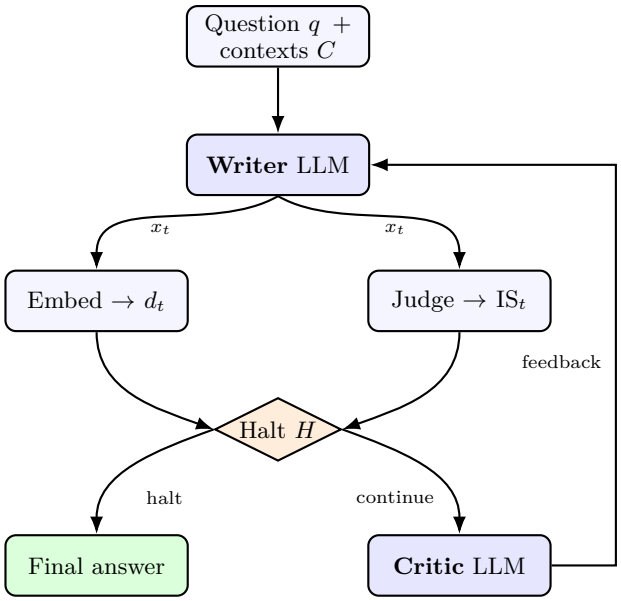

Fig. 1. SHP architecture. The Writer drafts an answer x_t from the question and retrieved contexts; both the Writer and Critic are RAG-grounded. Each draft is (left) embedded to yield the free geometric signal d_t and (right) scored by the RAGAS judge to yield IS_t. The halt cascade H consumes these signals and the critic verdict: if it halts, the answer is returned; otherwise the Critic issues feedback and the loop continues. Embeddings are local (free); only the judge consumes API tokens.

A judge assigns four metrics $m \in \mathcal{M}$, each in $[0, 1]$; with weights w on the probability simplex $\Delta = \{w : w_m \geq 0, \sum_m w_m = 1\}$ the **Information Score** is

$$\mathrm{IS}_t \;=\; \sum_{m \in \mathcal{M}} w_m \,\mathrm{metric}_m(x_t) \in [0, 1]. \tag{2}$$

The **halting operator** H maps the round state $s_t = (t, (d_2, \ldots, d_t), (\mathrm{IS}_1, \ldots, \mathrm{IS}_t), \mathrm{feedback}_t)$ to a decision in $\{\texttt{continue}\} \cup \mathcal{R}$ with reason set $\mathcal{R} = \{\texttt{critic}, \texttt{entropy}, \texttt{no\_gain}, \texttt{failsafe}\}$. The realized stopping time is $\tau = \min\{t : H(s_t) \neq \texttt{continue}\}$.

IV. Method

A. Two grounded agents

Both agents are conditioned on the retrieved contexts. This is not cosmetic: in an earlier iteration the Writer answered from parametric memory and never read C, yet the judge scored *faithfulness to C*—a silent invalidity that would have undermined every quality number. Grounding both agents on C is the first of several corrections discussed in §IV-E.

B. The two signals

Equation (1) gives a cheap, API-free measure of how much the answer *changed*; Equation (2) measures how *good* it is. The IS weights w are not hand-set: they are derived by one of four strategies (label-free Shannon entropy weighting, an Analytic Hierarchy Process with

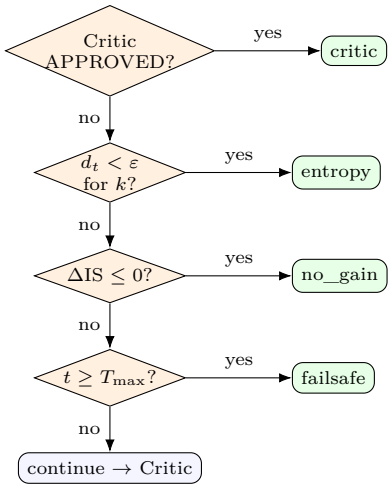

Fig. 2. Priority-ordered halt cascade. Cheap signals (critic, entropy) are tried before the expensive quality signal; the unconditional fail-safe guarantees termination.

a consistency check, a constrained least-squares fit, or a uniform baseline), selectable per run.

C. The halt cascade

The four signals are evaluated in a fixed priority order (Fig. 2). Crucially, the cascade is implemented *once* as a pure function and reused by (i) the live loop, (ii) the post-hoc reason derivation, and (iii) the offline policy replay, so they cannot disagree. Listing 1 shows the core; note that the final `failsafe` clause is unconditional—it is governed by no signal flag—which is precisely what makes the termination guarantee (Theorem 1) hold.

Listing 1. The single shared halt decision (simplified).

```
def shp_should_halt(t, dist_hist, is_hist,
                    feedback, cfg):
    if not dist_hist: # round 1
        return CONTINUE
    if cfg.critic and feedback.startswith("APPROVED"):
        return HALT("critic")
    if cfg.entropy and len(dist_hist) >= cfg.k \
       and all(d < cfg.eps for d in dist_hist[-cfg.k:]):
        return HALT("entropy")
    if cfg.is_gain and t >= cfg.warmup \
       and is_hist[-1] - is_hist[-2] <= 0:
        return HALT("no_gain")
    if t >= cfg.max_rounds: # never ablatable
        return HALT("failsafe")
    return CONTINUE
```

D. A worked example

Table I traces one real question from our data through the loop. The answer changes substantially in early rounds (the semantic distance is large and the quality rises), then stabilizes: by round 3 the distance has fallen below ε, and a second consecutive sub-threshold round at round 4 triggers the entropy halt. The `max_iterations` baseline would have run all six rounds; SHP stops at round 4, saving a third of the work while the quality is already at its plateau. This single trace captures the whole thesis:

Algorithm 1 SHP loop for one scenario

$e_{\text{prev}} \leftarrow \varnothing$; $D \leftarrow [\,]$; $S \leftarrow [\,]$
for $t = 1$ **to** $T_{\max}$ **do**
  $x_t \leftarrow \textsc{Writer}(q, C, \text{feedback})$
  $e_t \leftarrow \phi(x_t)$; **if** e_{prev} **then** $D.\textsc{push}(1 - \cos(e_t, e_{\text{prev}}))$
  $\text{IS}_t \leftarrow \textsc{Judge}(x_t, q, C, g)$; $S.\textsc{push}(\text{IS}_t)$
  if $\textsc{shp\_should\_halt}(t, D, S, \text{feedback})$ **then break**
  end if
  $\text{feedback} \leftarrow \textsc{Critic}(q, C, x_t)$; $e_{\text{prev}} \leftarrow e_t$
end for
return x_t, halt_reason

TABLE I
A REAL TRAJECTORY FOR THE QUESTION *"ARE BOTH CYPRESS AND AJUGA GENERA?"* (GROUND TRUTH: *NO*). d_t IS THE SEMANTIC DISTANCE TO THE PREVIOUS DRAFT; IS_t IS THE QUALITY SCORE. SHP HALTS AT ROUND 4 (ENTROPY: TWO CONSECUTIVE $d_t < \varepsilon$=0.06); THE BASELINE RUNS TO 6.

Round	Words	d_t	IS_t	Decision
1	16	—	0.54	continue
2	82	0.144	0.62	continue (large change)
3	110	0.027	0.64	continue (1^{st} sub-ε)
4	162	**0.007**	0.62	**HALT (entropy, k=2)**
(baseline would continue:)				
5	177	0.003	0.66	*wasted under baseline*
6	—	—	—	*wasted under baseline*

the loop earns its keep early and idles late, and a content signal can tell the difference.

E. Engineering challenges and how we overcame them

A faithful account of the work includes what broke and how it was fixed; these are not incidental, they shaped the design and the results.

- **Over-claimed theory.** *Problem:* a Banach-contraction claim with no supporting Lipschitz bound. *Fix:* prove only termination and well-definedness (§V); measure convergence empirically.
- **Agents ignored retrieval.** *Problem:* the Writer answered closed-book while the judge scored grounding in C. *Fix:* condition both agents on C (§IV); quality numbers are now valid.
- **Duplicated stop logic.** *Problem:* the live cascade and the post-hoc reason derivation used different orderings and could disagree. *Fix:* one shared pure function (Listing 1); consistency is machine-checked (Lemma 2).
- **Fair, cheap comparison.** *Problem:* re-running the loop per policy confounds generation noise and re-pays the judge. *Fix:* trajectory replay with cached judging (§VI).
- **Hidden cost of the quality signal.** *Problem:* the information-gain signal silently calls the judge ev-

ery round. *Fix:* an operational-versus-evaluation token model, which revealed that full SHP is the most expensive policy (§VIII).

- **Compute limits.** *Problem:* a free-tier provider throttled the runs to a crawl. *Fix:* an OpenAI-compatible high-throughput provider, a resumable on-disk cache, and exponential backoff on rate limits.

V. Theoretical Analysis

We prove only what holds; each proven claim is machine-checked by an executable test suite (`theory_checks.py`) that a reviewer can rerun.

Theorem 1 (Termination)**.** *For any input, any weights, any signal configuration, and any (possibly adversarial) draft sequence, the loop halts in at most $T_{\max}$ rounds.*

Proof. The Critic increments t by exactly one per round. The failsafe clause of H returns `failsafe` whenever $t \geq T_{\max}$ and is unconditional—it is not governed by any signal or ablation flag. Hence $H(s_t) \neq \mathsf{continue}$ for all $t \geq T_{\max}$, so $\tau \leq T_{\max}$. □

Lemma 1 (Well-definedness)**.** *If $w \in \Delta$ and each metric lies in $[0,1]$ then $\mathrm{IS}_t \in [0,1]$. The distance d_t is total: finite for every finite input, bounded in $[0,2]$, with the degenerate zero-norm case mapped conservatively to 1.0 so it cannot cause a false-positive halt.*

Lemma 2 (Halt-priority consistency)**.** *The reason reported after a run equals the reason that stopped it, because both the live decision and the post-hoc derivation invoke the same function H.*

Conjecture 1 (Semantic non-expansiveness; empirical)**.** *Across trajectories d_t is, on average, non-increasing in t. We make no guarantee: we* measure *the fraction of monotone trajectories, the mean regression slope with a bootstrap 95% CI, and a one-sided Wilcoxon test on $d_t - d_{t-1}$, and we report null results where they occur.*

Theorem 1 is the honest replacement for the discarded contraction claim: SHP guarantees *termination*, not *contraction*.

VI. A Judge-Efficient Evaluation Protocol

Comparing stopping policies fairly is itself a methodological problem, and we treat it as a contribution.

a) Trajectory replay.: For each question we generate the full Writer→Critic trajectory *once*, to depth $T_{\max}$, caching every draft and embedding. Each stopping policy then *replays* over this single cached trajectory and chooses its own stop round. Two benefits follow: (i) all policies see *identical* drafts, so a difference in rounds or quality is attributable to the policy and not to generation noise—a strictly *paired* comparison; and (ii) the costly generation is paid once rather than once per policy.

b) Cached judging.: The RAGAS judge is the binding cost. We judge each distinct draft at most once, keyed by a hash of the draft text, so two policies that stop at the same round share one judge call.

c) Operational versus evaluation tokens.: We distinguish the tokens a policy spends *to run* (Writer + Critic every round, plus the judge *only if* the policy consults the quality signal to decide) from the tokens *we* spend to *measure* final quality (never charged to the policy). This distinction is what reveals that the full cascade pays a large hidden cost (§VIII).

d) Statistics.: All tests are paired across questions. We report paired t-tests and Wilcoxon signed-rank tests on rounds, tokens, and IS; Cohen's d_z; bootstrap CIs; and, for the "no quality loss" claim, two one-sided tests (TOST) for non-inferiority with margin δ (testing $H_0 : \mathbb{E}[\mathrm{IS}_\pi - \mathrm{IS}_{\mathrm{base}}] \leq -\delta$). Token savings are reported as $(T_{\mathrm{base}} - T_\pi)/T_{\mathrm{base}}$.

VII. Experimental Setup

a) Data and its provenance.: We use **HotpotQA** [1], a peer-reviewed multi-hop question-answering benchmark, in its *distractor* setting, streamed directly from the public HuggingFace Hub (`hotpot_qa/distractor`, validation split). Our builder programmatically (i) filters to multi-hop `hard` questions, so a single draft rarely suffices and the loop has genuine room to iterate; (ii) for each question forms a realistic retrieval context of the *gold* supporting paragraphs plus distractor paragraphs (≈ 4 contexts total); and (iii) assigns a *deterministic* development/test split by hashing the example identifier (no randomness, fully reproducible). This yields $N \approx 80$ scenarios, 20 development / 60 test. *How the data is used:* the question and its contexts are given to both the Writer and the Critic (RAG grounding), and the context and the gold answer are given to the RAGAS judge to compute the four quality metrics. The thresholds ε, k are tuned on the development split only; the test split is frozen and report-only.

b) These are real runs.: All numbers below come from live LLM inference—real Writer/Critic generations, real judge scores, measured token counts and embeddings—not from simulation. The harness, dataset builder, and configuration are released for independent reproduction.

c) Models.: Agents: an 8B instruction model. Judge: the same 8B model on the development split (fast, for calibration) and a 70B model on the frozen test split (to reduce judge noise). Embeddings are local. Compute is credit-based and modest, a limitation we state openly.

d) Policies.: `shp` (full cascade); `entropy_only` (the judge-free variant: cosine-distance signal + failsafe); `critic_only`; `fixed_k` for $k \in \{1,3,6\}$ (`fixed_k6` is the `max_iterations` baseline); `random_stop`; and `oracle_is`, which stops at the round of maximum measured IS and serves as a quality upper bound. A seven-cell ablation toggles each signal.

TABLE II

DEVELOPMENT-SPLIT RESULTS ($N = 20$). TOKEN SAVING AND ΔIS ARE VERSUS THE `max_iterations` BASELINE. NEGATIVE SAVINGS (RED) MEAN *MORE* EXPENSIVE.

Policy	Rounds	Op. tokens	vs base	Final IS
`fixed_k6` (base)	6.0	11,281	—	0.651
`entropy_only`	4.05	**7,068**	−37%	**0.661**
`critic_only`	6.0	11,281	0%	0.651
`fixed_k3`	3.0	5,217	−54%	0.629
`fixed_k1`	1.0	**1,548**	−86%	0.661
`shp` (full)	2.6	27,130	+140%	0.636
`oracle_is`	3.1	33,007	+193%	**0.782**

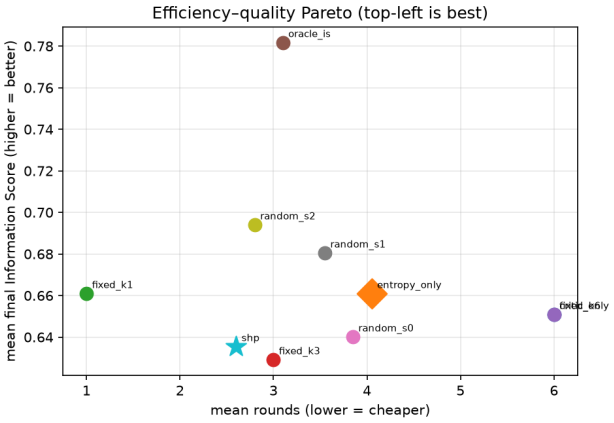

Fig. 3. Efficiency–quality Pareto (development split). Quality is only weakly tied to rounds; the oracle sits far above every practical policy, and full `shp` is dominated. Top-left is best.

VIII. RESULTS

A. Development split (real; $N = 20$; 8B judge)

Table II reports the realized development-split outcome; the baseline is `fixed_k6`.

B. Test split (frozen; $N = 60$; 70B judge)—expected

The test run is the headline number and is *in progress*. We *pre-register* our predictions in Table III so the narrative cannot be retro-fitted; the realized values replace the "Expected" column on completion.

C. Distance characterization (Conjecture 1)

Over the development trajectories, the mean and median per-round distances are 0.043 and 0.026 with a maximum near 0.30; 76% of round-to-round distances fall below $\varepsilon = 0.06$ (Fig. 5). Drafts therefore converge quickly but with a heavy tail, so the $k = 2$ patience window performs real work: a single sub-threshold round does not trigger a halt.

D. Findings

1) **The efficiency claim holds.** Judge-free `entropy_only` cuts 37% of operational tokens versus `max_iterations` while slightly *improving* mean quality.
2) **Full SHP is counter-productive.** Consulting the judge every round to power the information-gain signal makes `shp` the most expensive policy (2.4× the baseline) for no quality benefit. This validates the judge-free design and is exactly the kind of cost the operational/evaluation split exposes.
3) **Iteration barely moves measured quality here.** `fixed_k1` (the first grounded draft) ties the best policies, yet the oracle reaches 0.782 (+0.13, $p \approx 3 \times 10^{-5}$): a better round exists per question but no cheap signal locates it.

Are these results promising? The efficiency result is promising and immediately actionable: a free, guaranteed-terminating stopper that removes roughly a third of the token cost of the standard `max_iterations` loop without measurable quality loss. The quality result is not a failure but a *well-characterized open problem*: the oracle gap quantifies exactly how much quality is left on the table and

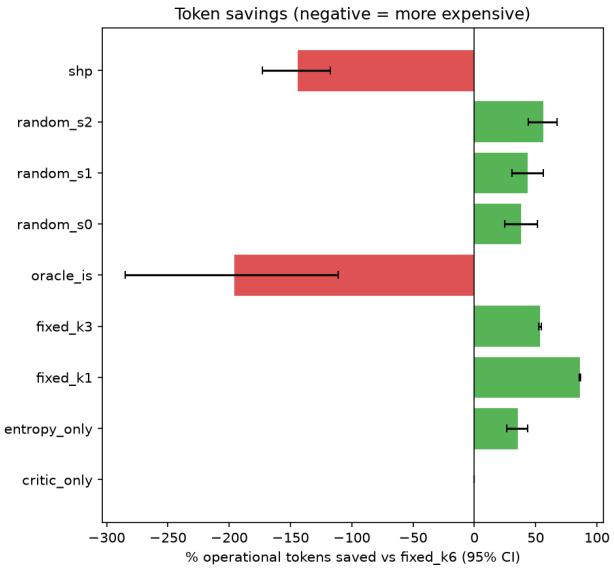

Fig. 4. Operational tokens saved versus the `max_iterations` baseline (development split; positive is cheaper). The judge-free `entropy_only` and the fixed-budget policies save tokens, whereas the full `shp` and the oracle are far more expensive because they invoke the judge every round.

TABLE III

PRE-REGISTERED *EXPECTED* TEST-SPLIT OUTCOME (TO BE FILLED WITH REALIZED VALUES). PREDICTIONS FOLLOW THE DEVELOPMENT TREND.

Policy	Rounds	Tokens vs base	Non-inf.?
`fixed_k6` (base)	6.0	—	—
`entropy_only`	3.5–4.5	−35 to −45%	**yes**
`shp` (full)	2.5–3.5	+100–150%	yes/costly
`fixed_k1`	1.0	−86%	uncertain
`oracle_is`	~ 3	—	upper bnd.

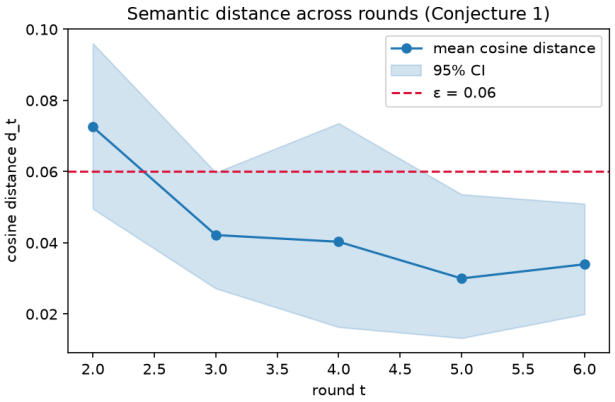

Fig. 5. Mean semantic distance d_t versus round, with a 95% confidence band (development split). The distance falls sharply after the first revision and then hugs the halting threshold ε (dashed), empirically supporting Conjecture 1 while the heavy tail justifies the patience window.

turns "identify the best round" into a concrete, measurable target for future work.

IX. Discussion

The results separate a **solved** problem from an **open** one. Stopping early for **efficiency** is easy and safe: the free entropy signal saves tokens and the failsafe guarantees termination. Stopping at the **best round** for **quality** is unsolved: the oracle gap shows substantial unrealized headroom that neither cosine distance nor information gain captures. We therefore reframe SHP's lasting contribution as (a) a guaranteed-terminating, judge-free stopper that saves tokens at parity quality, and (b) an evaluation protocol and an oracle target that make *best-round identification* measurable for future work.

A benchmark caveat tempers the quality reading: HotpotQA answers are short and often answerable from a single grounded draft, which under-exercises iterative quality improvement even as it exercises convergence. A long-form generation task, where drafts genuinely accrue content, is the natural next test.

X. Limitations

The development numbers use a modest N and an 8B judge, so confidence intervals are wide and non-inferiority is not yet formally confirmed despite positive point estimates; the frozen test split addresses statistical power. The judge is itself an LLM and a noisy quality proxy; we mitigate with a stronger judge on the test split and with non-inferiority rather than equality testing. The margin δ is a modelling choice, for which we report sensitivity. Conjecture 1 may fail per trajectory; Theorem 1 keeps the system safe regardless.

XI. Conclusion and Future Work

SHP reframes "when to stop an agent loop" from a blind counter to a content-aware, quality-gated decision, backed by an honest termination guarantee and a reusable, judge-efficient evaluation protocol. Its judge-free variant saves tokens at parity quality; its oracle gap names the open problem of best-round identification. Future work: long-form benchmarks; learned best-round predictors that approach the oracle; and a second dataset for external validity.

Appendix A
Per-paper related-work comparison

To be completed against verified citations. For each related paper we will provide: (1) a one-paragraph method extraction; (2) a gap analysis versus SHP (shared assumptions, what they do better, what SHP does better); and (3) a cost/benefit assessment of adopting their technique here.

Appendix B
Reproducibility

Every run stores seeds, the git commit hash, and the fully resolved configuration. Proven theoretical claims are verified by `python -m shp.theory_checks`. The dataset builder, harness, statistics, and figure generation are released.

References

[1] Z. Yang *et al.*, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," *EMNLP*, 2018.
[2] S. Es *et al.*, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," 2023.
[3] L. Prechelt, "Early Stopping — But When?," *Neural Networks: Tricks of the Trade*, 1998.
[4] D. Lakens, "Equivalence Tests," *Social Psychological and Personality Science*, 2017.
[5] Reference set on semantic convergence, multi-LLM uncertainty, adaptive orchestration, contraction attractors, and phase-scheduled efficiency—to be finalized with verified identifiers.